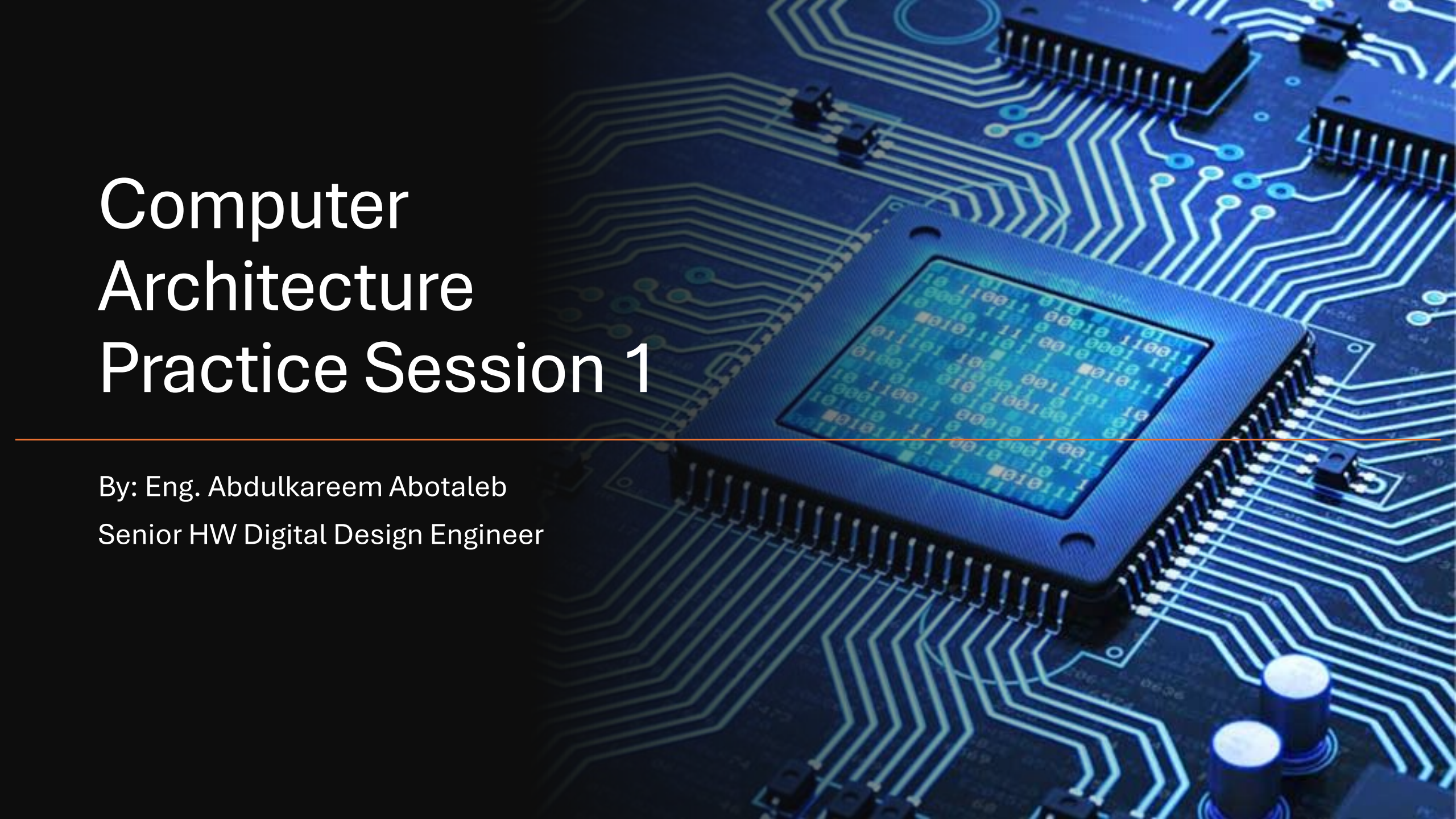
Computer Architecture Practice Session 1



By: Eng. Abdulkareem Abotaleb

Senior HW Digital Design Engineer

Agenda

1. MIPS Basic Architecture

2. MIPS Hierarchy

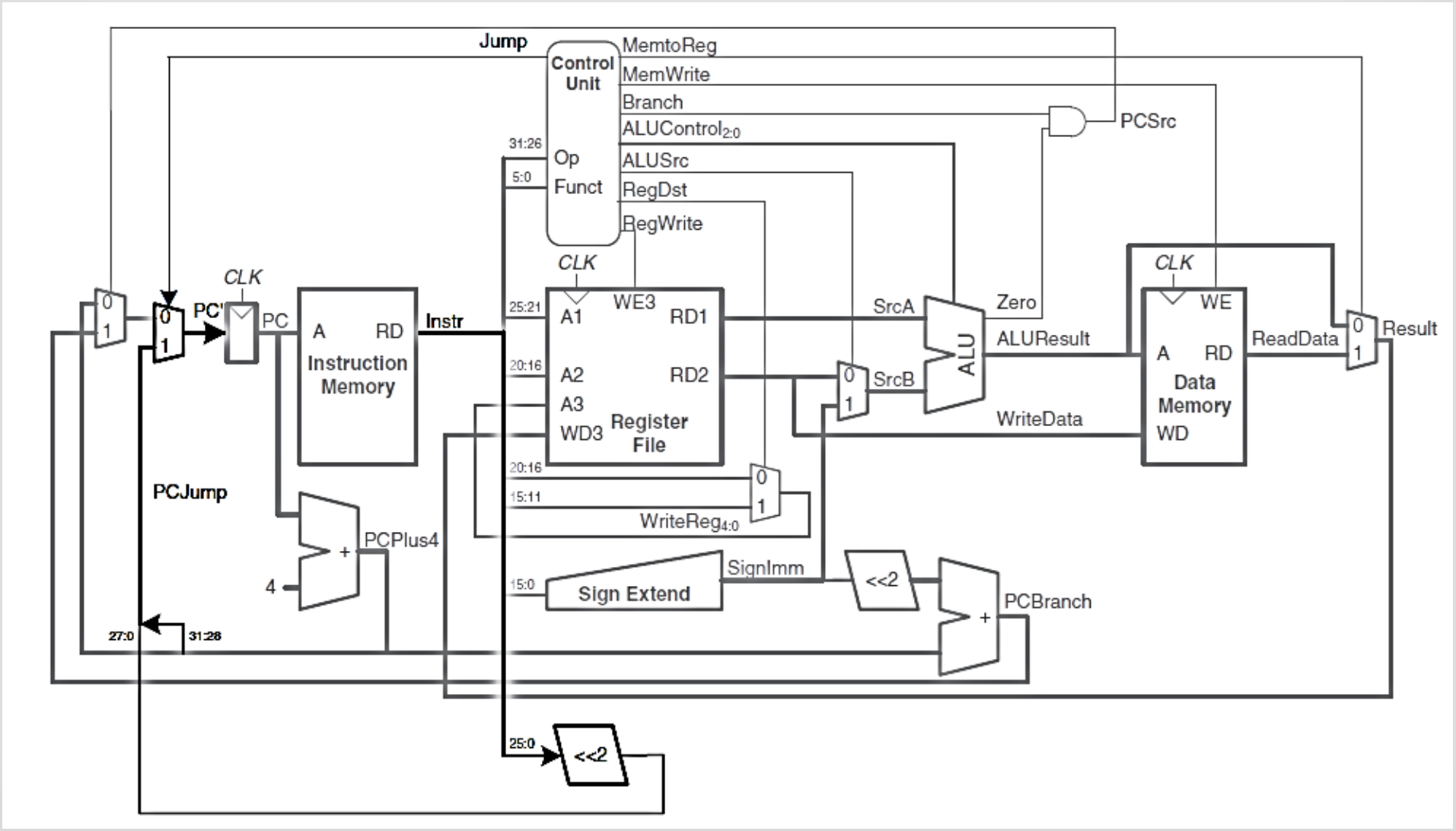
3. Component Implementation

4. Testbench

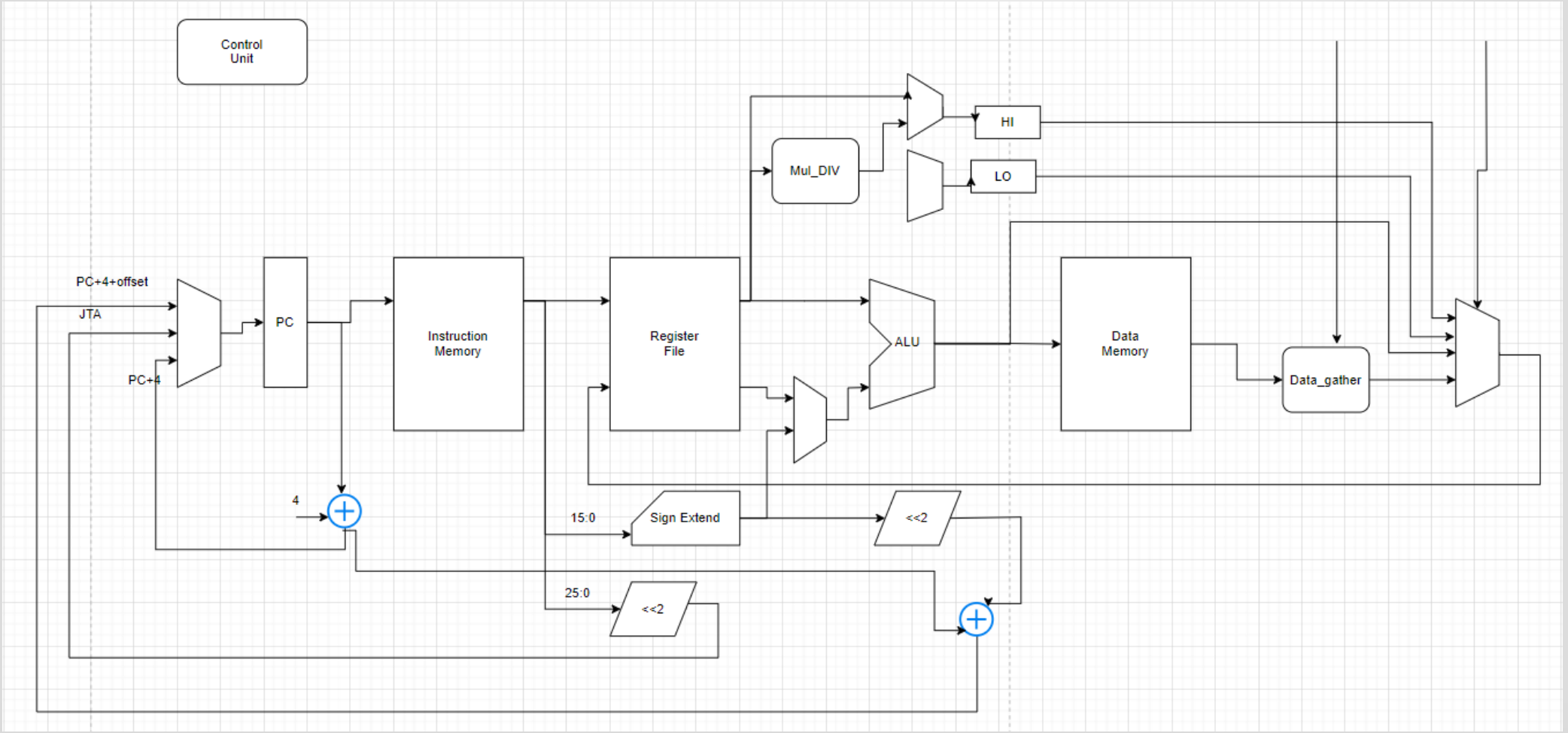
5. Synthesis

1. MIPS Basic Architecture

1. MIPS Basic Architecture



1. MIPS Basic Architecture



2. MIPS Hierarchy

2. MIPS Hierarchy

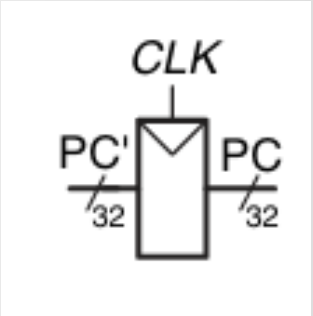
- mips_tbench
  - mips_top_inst
    - pc_reg_inst
    - instr_mem
    - reg_file_inst
    - alu_inst
    - data_mem
    - ctrl_unit_inst

3. Component Implementation

2. Component Implementation

- PC Implementation:
  Reset is asynch, active-low.

```
module pc_reg ( input clk, input rst,
   input [31:0] pc_next,
   output reg [31:0] pc
);
   always @(posedge clk or negedge rst) begin
      if(~rst)
         pc <= 0;
      else begin
         pc <= pc_next;
      end
   end
endmodule
```



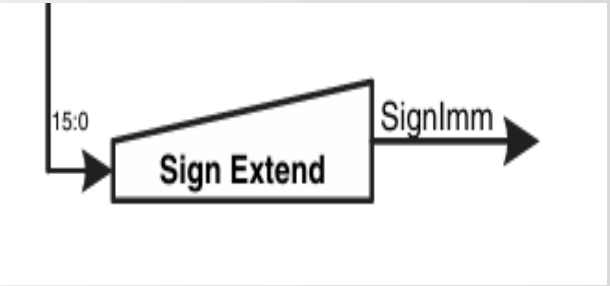
2. Component Implementation

- Sign Extension Implementation:
  We will repeat bit#15 of the immediate, then concatenate with the immediate value from the instruction.

```
module sign_extend (
  //input
  input  [15:0] instruction_imm,
  //output
  output  [31:0] sign_imm
);

assign  sign_imm = {{16{instruction_imm  [15]}}, instruction_imm  } ;

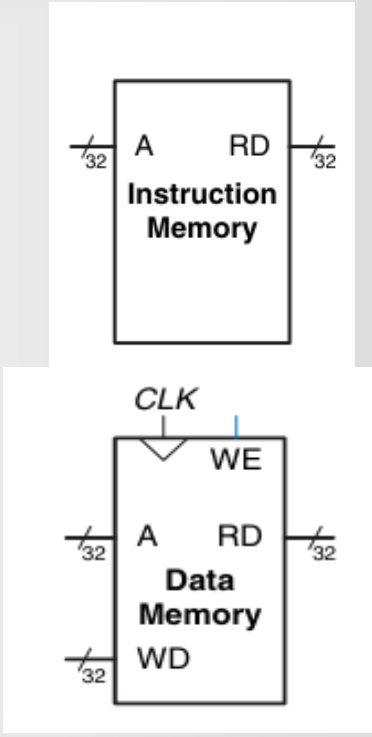
endmodule
```



2. Component Implementation

- Instruction/Data Memory:

```
module ram_memory #(parameter depth = 1024,  parameter width = 8) (
   //input
   input clk,     input wr_en,
   input [$clog2(depth)-1:0] addr,
   input [(4*width)-1:0] wr_data,
   //output
   output [(4*width)-1:0] rd_data
);
reg [width-1:0] mem [0:depth-1];
initial begin
                for(integer i = 0; i < depth; i = i+1 )
                     mem [i] <= 0;
end
always @(posedge(clk)) begin
                if(wr_en)
                     {mem [addr+3], mem [addr+2], mem [addr+1], mem [addr]} <= wr_data;
end
assign  rd_data = {mem [addr+3], mem [addr+2], mem [addr+1], mem [addr]};
endmodule
```



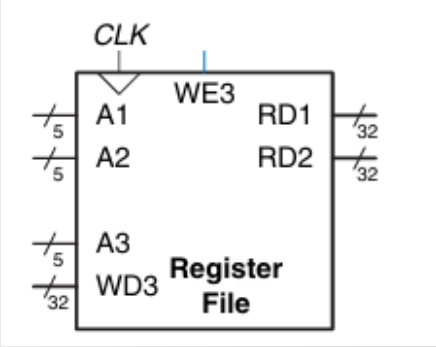
2. Component Implementation

- Register File:

```
module register_file (
  //input
  input wire          clk,
  input wire          wr_en3,
  input wire [4:0]    rd_addr1,
  input wire [4:0]    rd_addr2,
  input wire [4:0]    wr_addr3,
  input wire [31:0]   wr_data3,
  //output
  output wire [31:0]   rd_data1,
  output wire [31:0]   rd_data2 ,
  output wire [3:0] s0_reg
);

reg [31:0]  register_file  [0:31];

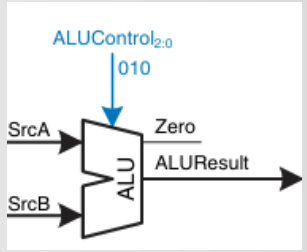
assign   rd_data1  =   (rd_addr1 != 0)? register_file[rd_addr1]:0;
assign   rd_data2  =   (rd_addr2 != 0)? register_file[rd_addr2]:0;
assign   s0_reg = register_file[16][3:0];
always @(posedge clk) begin
     if (wr_en3)
          register_file[wr_addr3] <= wr_data3;
end
endmodule
```



2. Component Implementation

- ALU:

```
module alu (
  //input
  input  wire  signed [31:0]  src_a,
  input  wire  signed [31:0]  src_b,
  input  wire  [2:0]  alu_control,
  //output
  output wire                 zero,
  output reg signed [31:0]  alu_result
);
always @(*) begin
    case(alu_control)
            3'b000 : alu_result = src_a & src_b  ;
            3'b001 : alu_result = src_a | src_b  ;
            3'b010 : alu_result = src_a + src_b  ;
            3'b110 : alu_result = src_a - src_b  ;
            3'b111 : alu_result = (src_a < src_b) ;
            default: alu_result = src_a;
    endcase
end
assign zero = ~|alu_result ;
endmodule
```



$F_{2:0}$	Function
000	A AND B
001	A OR B
010	A + B
011	not used
100	A AND $\overline{B}$
101	A OR $\overline{B}$
110	A – B
111	SLT

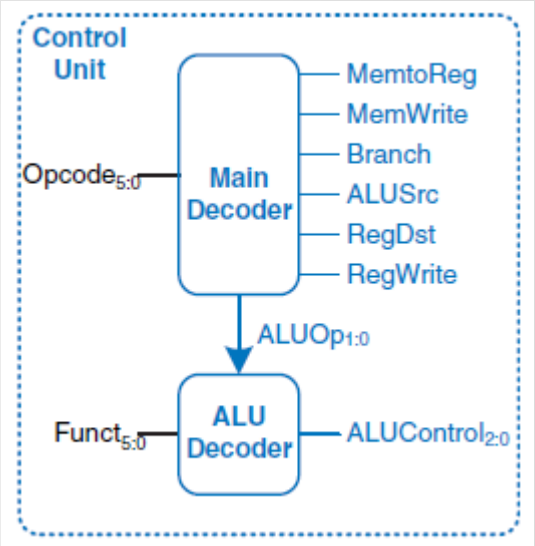
2. Component Implementation

- Control Unit:

```
module ctrl_unit (
  // inputs
  input  wire [5:0]  funct,
  input  wire [5:0]  opcode,
  // outputs
  output reg         mem_to_reg,
  output reg         mem_write,
  output reg         branch,
  output reg  [2:0]  alu_control,
  output reg         alu_src,
  output reg         reg_dst,
  output reg         reg_write,
  output reg         jump
);

reg [1:0] alu_op;
...

endmodule
```



Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
addi	001000	1	0	1	0	0	0	00	0
j	000010	0	X	X	X	0	X	XX	1

2. Component Implementation

- Control Unit:

Instruction	Opcode	RegWrite	RegDst	ALUSrc	Branch	MemWrite	MemtoReg	ALUOp	Jump
R-type	000000	1	1	0	0	0	0	10	0
lw	100011	1	0	1	0	0	1	00	0
sw	101011	0	X	1	0	1	X	00	0
beq	000100	0	X	0	1	0	X	01	0
addi	001000	1	0	1	0	0	0	00	0
j	000010	0	X	X	X	0	X	XX	1

```
always @(*) begin
  case (opcode)
    6'b000000  : begin reg_write=1; reg_dst=1; alu_src=0; branch=0; mem_write=0; mem_to_reg=0; alu_op=2'b10; jump=0; end //R-type
    6'b100011  : begin reg_write=1; reg_dst=0; alu_src=1; branch=0; mem_write=0; mem_to_reg=1; alu_op=2'b00; jump=0; end // lw
    6'b101011  : begin reg_write=0; reg_dst=0; alu_src=1; branch=0; mem_write=1; mem_to_reg=0; alu_op=2'b00; jump=0; end // sw
    6'b000100  : begin reg_write=0; reg_dst=0; alu_src=0; branch=1; mem_write=0; mem_to_reg=0; alu_op=2'b01; jump=0; end // beq
    6'b001000  : begin reg_write=1; reg_dst=0; alu_src=1; branch=0; mem_write=0; mem_to_reg=0; alu_op=2'b00; jump=0; end // addi
    6'b000010  : begin reg_write=0; reg_dst=0; alu_src=0; branch=0; mem_write=0; mem_to_reg=0; alu_op=2'b00; jump=1; end // j
    default      : begin reg_write=1; reg_dst=1; alu_src=0; branch=0; mem_write=0; mem_to_reg=0; alu_op=2'b00; jump=1; end
  endcase
end
```

2. Component Implementation

- Control Unit:

```
always @(*) begin
    casex({alu_op ,funct})
            8'b00_xxxxxx  : alu_control = 3'b010;  //add
            8'bx1_xxxxxx  : alu_control = 3'b110;  //subtract
            8'b1x_100000 : alu_control = 3'b010;  //add
            8'b1x_100010 : alu_control = 3'b110;  //subtract
            8'b1x_100100 : alu_control = 3'b000;  //and
            8'b1x_100101 : alu_control = 3'b001;  //or
            8'b1x_101010 : alu_control = 3'b111;   //set less than
            default          : alu_control = 3'b010;  //default
    endcase
end
```

ALUOp	Funct	ALUControl
00	X	010 (add)
X1	X	110 (subtract)
1X	100000 (add)	010 (add)
1X	100010 (sub)	110 (subtract)
1X	100100 (and)	000 (and)
1X	100101 (or)	001 (or)
1X	101010 (slt)	111 (set less than)

4. Testbench

4. Testbench

- Direct Testcase (1):

addi $s0, $0, 0x0AF0 //2800	20100af0
addi $s1, $0, 0x04B0 //1200	201104b0
add $s2, $s0, $s1	02119020
addi $t0, $0, 0x0FA0 //Result	20080fa0
beq $s2, $t0, NEXT1	12480001
j ERROR	08000016
NEXT1: sub $s3, $s0, $s1	02119822
addi $t1, $0, 0x0640 //Result	20090640
beq $s3, $t1, NEXT2	12690000
j ERROR	08000016
NEXT2: and $s4, $s0, $s1	0211a024
addi $t2, $0, 0x00B0 //Result	200a00b0
beq $s4, $t2, NEXT3	128a0000
j ERROR	08000016

NEXT3: or $s5, $s0, $s1	0211a825
addi $t3, $0, 0x0EF0 //Result	200b0ef0
beq $s5, $t3, NEXT4	12ab0000
j ERROR	08000016
NEXT4: addi $t4, $0, 0x0064	200c0064
sw $s0, 50($t4)	ad900032
lw $t5, 50($t4)	8d8d0032
beq $s0, $t5, DONE	120d0002
ERROR: addi $s0, $0, 0XDEAD	2010dead
j SKIP	08000018
DONE: addi $s0, $0, 0XD08E	2010d08e
SKIP: nop	00000000

4. Testbench

- Randomized Testcase:
  Build enum for the main parts of your instruction to be randomized later.

```
////////////////////////////////////////////////////////////////////////////////
typedef enum logic [1:0] {R_TYPE, I_TYPE, J_TYPE} instr_type_e;

typedef enum logic [5:0] {r_type=0, j=2, beq=4, addi=8, lw=35, sw=43} opcode_e;

typedef enum logic [5:0] {add=32, sub=34, andd=36, orr=37, slt=42} funct_e;

typedef enum logic [4:0] {rs_t0=8,rs_t1,rs_t2,rs_t3,rs_t4,rs_t5,rs_t6,rs_t7=15,
rs_s0=16,rs_s1,rs_s2,rs_s3,rs_s4,rs_s5,rs_s6,rs_s7,rs_t8,rs_t9=25} rs_e;

typedef enum logic [4:0] {rt_t0=8,rt_t1,rt_t2,rt_t3,rt_t4,rt_t5,rt_t6,rt_t7=15,
rt_s0=16,rt_s1,rt_s2,rt_s3,rt_s4,rt_s5,rt_s6,rt_s7,rt_t8,rt_t9=25} rt_e;

typedef enum logic [4:0] {rd_t0=8,rd_t1,rd_t2,rd_t3,rd_t4,rd_t5,rd_t6,rd_t7=15,
rd_s0=16,rd_s1,rd_s2,rd_s3,rd_s4,rd_s5,rd_s6,rd_s7,rd_t8,rd_t9=25} rd_e;
////////////////////////////////////////////////////////////////////////////////
```

4. Testbench

- Randomized Testcase:

  Then, construct each instruction type.

```
typedef struct packed {
        opcode_e opcode;
        rs_e rs;
        rt_e rt;
        rd_e rd;
        logic [4:0] shamt;
        funct_e funct;
} R_TYPE_e;

typedef struct packed {
        opcode_e opcode;
        rs_e rs;
        rt_e rt;
        logic signed [15:0] immediate;
} I_TYPE_e;

typedef struct packed {
        opcode_e opcode;
        logic signed  [25:0] immediate;
} J_TYPE_e;
```

4. Testbench

- Randomized Testcase:

  Then, randomize each type, and do your constraints.

```
class transaction;
        rand logic [31:0] instruction;
        rand R_TYPE_e       R_TYPE_instruction;
        rand I_TYPE_e       I_TYPE_instruction;
        rand J_TYPE_e       J_TYPE_instruction;
        rand instr_type_e   option;

        constraint instr_constr {
            if(option==R_TYPE) { instruction==R_TYPE_instruction; }
            else if(option==I_TYPE) { instruction==I_TYPE_instruction; }
            else if(option==J_TYPE) { instruction==J_TYPE_instruction; }
        }

        constraint instr_dist_constr {
            option dist {R_TYPE:=60, I_TYPE:=40, J_TYPE:=10};
        }
endclass
```

4. Testbench

- Randomized Testcase:

  After that, build your testbench, creating the required clk, and write the randomized instruction inside a file to be put initially inside the instruction mem.

```
module mips_tbench();
    ////////////////////////////////////////
    parameter clk_period=10;
    logic clk;
    initial begin
            clk=0;
            forever
            #(clk_period/2) clk=~clk;
    end
    ////////////////////////////////////////
    transaction t;
    task write_instruction_file();
            fd=$fopen("mem.dat","w");
            for(int i=0;i<1024;i=i+1) begin
                assert(t.randomize() );
                $fdisplay(fd,"%h",t.instruction[7:0]);
                $fdisplay(fd,"%h",t.instruction[15:8]);
                $fdisplay(fd,"%h",t.instruction[23:16]);
                $fdisplay(fd,"%h",t.instruction[31:24]);
            end
            $fclose(fd);
     endtask
      initial $readmemh("mem.dat", mips_top_inst.instruction_mem_ins.mem);
endclass
```

5. Synthesis

5. Synthesis

- Edit XDC file:
- Create clock at clk port with 100 MHz.

```
## This file is a general .xdc for the ZYBO Rev B board
## To use it in a project:
## - uncomment the lines corresponding to used pins
## - rename the used signals according to the project


##Clock signal
set_property -dict { PACKAGE_PIN L16   IOSTANDARD LVCMOS33 } [get_ports { clk }]
create_clock -add -name sys_clk_pin -period 20.00 -waveform {0 10} [get_ports { clk }]
```

Thank You